PRACTICAL PROJECT HANDOVER

Laravel Product Management API

Laravel 12, PHP, MySQL, Redis cache, service classes, REST endpoints, and feature testing.

A plain-language technical guide for explaining, operating, and testing the implementation.

Project: practice-app · Prepared: 9 October 2026

Contents

1. What the application does
2. Architecture and request flow
3. Redis and Laravel configuration
4. Files and responsibilities
5. Service class and cache behavior
6. REST API reference
7. Cache invalidation
8. Testing through the browser
9. Testing the API and automated tests
10. Verifying Redis, TTL, hits, misses, and persistence
11. Common problems
12. Quick explanation for another developer

1. What the application does

This project has a Products page and a JSON API. The browser page at `/products` calls the API; the API reads and writes product records in MySQL. Laravel's cache layer stores frequently requested product data in Redis for ten minutes.

Browser /products → Laravel API controller → ProductService → MySQL
↕ Redis cache

The service class keeps product operations and caching out of the controller. The controller handles HTTP input, validation, and response codes. The service handles database work and cache rules.

> **Current project status:** Redis is configured as Laravel's cache store, Memurai (Redis-compatible server) runs on Windows, the product API routes are registered, and the product feature tests exist.

Important access note

The current product API routes are public: they do not require a Sanctum token or login. This is convenient for local browser testing. For a real private catalog, add authentication and authorization middleware before deployment.

2. Architecture and request flow

1. The browser sends a request such as `GET /api/products`.
2. `routes/api.php` routes it to `ProductController`.
3. The controller validates incoming fields when needed, then calls `ProductService`.
4. The service checks Redis. If a matching cached value exists, it returns that value. Otherwise it queries MySQL and stores the result in Redis.
5. The controller returns JSON with an HTTP status code.

Why have a service class?

A controller should be easy to read. If SQL queries, cache keys, expiry rules, and invalidation were all inside the controller, those rules would be duplicated across endpoints. `ProductService` centralizes those business rules so they can be changed and tested in one place.

What is a cache hit or miss?

Term	Meaning	Example
Cache miss	The requested key is not in Redis. Laravel gets data from MySQL and saves it in Redis.	First request for the active product list.
Cache hit	The requested key is already in Redis. Laravel returns that cached value without running the product query again.	Refreshing the same product list shortly afterward.
Expiration / TTL	Time to live: how many seconds remain before Redis expires a key.	A new cached product list starts near 600 seconds.

3. Redis and Laravel configuration

Windows Redis-compatible server

Memurai is the Redis-compatible Windows service used in this setup. Check that Windows reports it as running:

```
Get-Service *memurai*
```

Expected service status: `Running`. The CLI executable is installed at `C:\Program Files\Memurai\memurai-cli.exe`. It is not on this machine's PATH, so call it by full path:

```
& "C:\Program Files\Memurai\memurai-cli.exe" ping
# Expected: PONG
```

Laravel environment values

In the project's `.env`, the cache-related values are:

```
CACHE_STORE=redis
REDIS_CLIENT=predis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
```

`predis/predis` is the PHP Redis client package. Laravel connects to Memurai through Predis. Do not share the rest of `.env`: it can contain database passwords, mail credentials, and application keys.

Laravel's `config/cache.php` selects the Redis cache connection. In `config/database.php`, the cache Redis connection uses database index `1` by default; the normal Redis connection uses database index `0`. That is why Redis CLI inspection uses `-n 1`.

Laravel also adds a prefix to keys to avoid collisions with other apps. In this project observed keys begin with `laravel-database-laravel-cache-`. Do not hard-code that prefix in application code.

Clear cached configuration after changing .env

```
php artisan config:clear
```

Restart the Laravel development server after configuration changes so the running PHP process uses the current settings.

4. Project files and their responsibilities

File	Purpose
`app/Models/Product.php`	Product model and allowed mass-assignment fields.
`database/migrations/*create_products_table.php`	Creates the MySQL products table.
`app/Services/ProductService.php`	Lists, searches, retrieves, creates, updates, deletes, caches, and invalidates product data.
`app/Http/Controllers/Api/ProductController.php`	Validates requests, calls the service, and returns JSON and HTTP status codes.
`routes/api.php`	Declares product API URLs.
`bootstrap/app.php`	Registers `routes/api.php` with the Laravel application.
`resources/views/products/api-manager.blade.php`	Browser UI at `/products`; uses JavaScript fetch requests to the API.
`tests/Feature/ProductApiTest.php`	Automated checks for API responses, validation, cache hit/miss, and invalidation.

Product table

The product record has: `id`, `name`, nullable `description`, decimal `price`, integer `stock` (default 0), boolean `is_active` (default true), and timestamps.

The model's fillable list contains `name`, `description`, `price`, `stock`, and `is_active`. This allows the validated fields to be passed safely to `Product::create()` and `$product->update()`.

5. ProductService and caching rules

The service uses Laravel's `Cache` facade with a `products` tag. `remember()` means: return an existing value if present; otherwise run the callback, save its result, and return it.

Data	Cache key	Lifetime
Active product list	`products:active:list`	10 minutes
Product details	`products:{id}:details`	10 minutes
Search results	`products:search:{sha1(normalized term)}`	10 minutes

Search text is trimmed and lowercased before its SHA-1 digest is used. The digest keeps the key predictable and avoids putting arbitrary search text directly into a key.

Service code pattern

```
return Cache::tags(['products'])->remember(
    'products:active:list',
    now()->addMinutes(10),
    fn () => Product::where('is_active', true)->get()
);
```

The individual lookup uses `Product::findOrFail($id)`, so an unknown ID becomes a 404 response. Search only includes active products and matches name or description.

Why cache tags?

A product write may affect the list, a detail page, and many search results. Laravel's product tag lets the service invalidate all of those related entries with one operation: `Cache::tags(['products'])->flush()`. This avoids having to know every search term that may already have been cached.

> **Tag support matters:** the Redis cache store supports tags. The array and file cache stores do not support this tagged-cache design. The feature test explicitly selects Redis for its cache operations.

6. REST API reference

Method	URL	Purpose	Success code
GET	`/api/products`	List active products	200
GET	`/api/products/{id}`	Get one product	200 (404 if missing)
GET	`/api/products/search?q=phone`	Search active products by name/description	200
POST	`/api/products`	Create a product	201
PUT/PATCH	`/api/products/{id}`	Update a product	200 (404 if missing)
DELETE	`/api/products/{id}`	Delete a product	200

Product JSON fields

```
{
  "name": "Wireless headphones",
  "description": "Over-ear headphones",
  "price": 49.99,
  "stock": 12,
  "is_active": true
}
```

`name` and `price` are required when creating. Price must be numeric and at least 0. Stock must be an integer of at least 0. `is_active` must be boolean. Update accepts only the supplied fields. Search requires `q`, a string no longer than 100 characters.

Laravel returns `422 Unprocessable Content` with validation errors for invalid JSON fields. Successful JSON responses use a `data` property, and create/update/delete responses also include a message.

7. Cache invalidation

Creating, updating, or deleting a product changes data that could appear in cached lists, details, and searches. After a successful write, the service flushes the `products` tag.

Action	Expected cache behavior
First list/search/detail GET	Miss: query MySQL, then store the result in Redis.
Repeat same GET within 10 minutes	Hit: return cached data.
Create product	Flush product tag; next list/search request sees the new product.
Update product	Flush product tag; next detail/list/search request sees new fields and terms.
Delete product	Flush product tag; next list/search excludes it, detail returns 404.
No writes for 10 minutes	Redis expires each key; the next request queries MySQL and caches fresh data again.

The product tag metadata may remain briefly after an individual data key expires. The important detail key is the one ending in `:products:active:list`; inspect its TTL rather than assuming every tag bookkeeping key has the same TTL.

8. Testing through the browser

1. Start the Laravel server and open `http://localhost:8000/products`.
2. Add one product, for example *Cache Test Product*, price `19.99`, stock `4`.
3. Confirm it appears in the catalog. Refresh the page and confirm it remains.
4. Search for *Cache Test* and confirm the product appears.
5. Edit the product name to *Updated Cache Test Product*, save, and confirm the changed name appears.
6. Search the new name. This checks that a previously cached search does not show stale results.
7. Delete the product and confirm the styled confirmation dialog. Check that it disappears from the list.

> **Browser limitation:** the page looks the same whether a response came from MySQL or Redis. Browser testing verifies behavior and invalidation, but by itself it cannot prove a cache hit. Use Redis key inspection and feature tests for that proof.

Browser DevTools → Network can show the requests to `/api/products` and their JSON responses. There is currently no custom response header that says “cache hit.”

9. API and automated testing

PowerShell manual API check

With `php artisan serve` running, use another PowerShell window:

```
$base = 'http://127.0.0.1:8000/api/products'
$created = Invoke-RestMethod -Method Post -Uri $base `
  -ContentType 'application/json' `
  -Body (@{
    name = 'Redis demo'
    description = 'Cache test product'
    price = 19.99
    stock = 4
    is_active = $true
  } | ConvertTo-Json)
$created
$id = $created.data.id

Invoke-RestMethod "$base"
Invoke-RestMethod "$base/$id"
Invoke-RestMethod "$base/search?q=Redis"

Invoke-RestMethod -Method Put -Uri "$base/$id" `
  -ContentType 'application/json' `
  -Body (@{ name = 'Updated Redis demo' } | ConvertTo-Json)

Invoke-RestMethod -Method Delete -Uri "$base/$id"
```

Expected: POST returns a new object (201), GET returns JSON (200), update shows the new name (200), and delete returns a success message (200). A missing detail after deletion should return 404.

Feature tests

```
php artisan test --filter=ProductApiTest
```

`ProductApiTest` uses an in-memory SQLite database so test product rows do not go into the regular MySQL database. It selects Redis as the cache store and clears the product tag around each test. The tests check active-only listing, search validation, create/update/delete, cache invalidation, and cache hit/miss events.

For the full project test suite:

```
php artisan test
```

10. Verifying Redis, expiry, and persistence

Check that a key is physically in Redis

Use Memurai CLI in PowerShell. The full executable path is required on this computer:

```
& "C:\Program Files\Memurai\memurai-cli.exe" -n 1 --scan --pattern "*products*"
```

A successful check previously returned keys like these:

```
laravel-database-laravel-cache-tag:products:entries
laravel-database-laravel-cache-6266bc2904f983980f02a3d69bd7b8985f76a5d0:products:active:list
```

The long prefix is generated from Laravel's app/cache prefixes. The part ending in `:products:active:list` is the cached active list. Redis stores Laravel-serialized data, so the value may not look like readable JSON.

Check its remaining expiry

```
& "C:\Program Files\Memurai\memurai-cli.exe" -n 1 TTL "PASTE_THE_FULL_ACTIVE_LIST_KEY_HERE"
```

The returned integer is remaining seconds. The observed value `486` meant 486 seconds remained from a 600-second (10-minute) lifetime. A TTL of `-2` means the key no longer exists; `-1` means it exists without an expiry, which would be unexpected for this key.

What “persistence” means here

This project verifies that cached values are stored in Redis and reused across separate HTTP requests while the Redis service is running. The 10-minute Redis TTL is verified through Redis CLI. Surviving a Redis/Memurai service restart is a separate disk-persistence setting (RDB/AOF) and is not established by Laravel's cache configuration alone. Do not claim restart survival unless Memurai's persistence settings are also configured and tested.

Proving a hit and a miss

The feature test fakes Laravel's `CacheMissed` and `CacheHit` events: the first request should miss and the repeated request should hit. The Redis CLI proves the key exists; the feature test is the direct hit/miss proof. No special badge is required on the frontend.

11. Common problems

Problem	Meaning / fix
`memurai-cli is not recognized`	The executable is not in PATH. Use `& "C:\Program Files\Memurai\memurai-cli.exe" ...`.
Redis connection refused	Check `Get-Service *memurai*`; start Memurai, check host `127.0.0.1` and port `6379`, then clear Laravel config and restart the app.
No product key appears	Visit `/products` or call `GET /api/products` first. A cache key is created when the cached operation is requested.
TTL command returns `-2`	The key expired, was invalidated by a product write, or the wrong Redis DB/prefix/key was used. Load the list again and scan again.
TTL starts under 600	Some seconds may pass between loading the list and checking the key. A result such as 486 is normal if about 114 seconds passed.
Class/method not found after code/config edits	Run `php artisan optimize:clear` if route/config/view caches are stale; restart the PHP server.
Cache tags not supported	Ensure `CACHE_STORE=redis`; this service deliberately uses Redis tags.

> **Safe inspection:** use `--scan` and `TTL`. Avoid `FLUSHALL` or broad key deletion; those commands can erase unrelated Redis data.

12. Quick explanation for another developer

“The Products browser page calls our Laravel JSON API. The API controller validates requests and delegates product work to `ProductService`. The service caches the active list, product detail, and search results in Redis for ten minutes with meaningful keys. On a cache miss it reads MySQL and stores the result; on a hit it returns Redis data. Product create/update/delete flushes the shared products cache tag so old list, detail, and search results are not returned. The API tests use an in-memory SQLite database and verify CRUD, validation, invalidation, and cache hit/miss events. We verified Redis storage with Memurai CLI: it showed the active-list key and a TTL counting down from 600 seconds.”

Short glossary

Term	Plain meaning
REST API	URLs that use HTTP methods to read or change data and return JSON.
Controller	Receives HTTP requests, validates input, calls app logic, returns a response.
Service class	Reusable place for product business logic and cache rules.
Redis	Fast in-memory key/value store used here for Laravel cache.
Cache key	Name used to find one cached value in Redis.
Invalidation	Removing old cached values after the underlying product data changes.
TTL	Seconds remaining before a cached key expires.
Feature test	Automated test that exercises an application request and checks its outcome.

This guide describes the current practice-app implementation. Check the actual files if the code changes after this handover.